10. Duties Service 구현 분석

개요

`DutiesService` 는 Lighthouse 검증자 클라이언트의 핵심 컴포넌트로, 모든 로컬 검증자들의 증명자(attester) 및 제안자(proposer) 듀티를 관리하는 역할을 담당합니다. 이 서비스는 비콘 노드로부터 정기적으로 듀티 정보를 폴링하여 최신 상태를 유지하며, 블록 생성을 위한 이벤트를 `BlockService` 에 전송하는 책임도 가지고 있습니다.

DutiesService 구조체 분석

핵심 데이터 구조

```
// validator_client/validator_services/src/duties_service.rs

pub struct DutiesService<S, T> {

/// 검증자 공개키를 각 에포크별 듀티에 매핑

pub attesters: RwLock<AttesterMap>,

/// 에포크를 해당 에포크의 모든 로컬 제안자들에 매핑

pub proposers: RwLock<ProposerMap>,

/// 검증자 인덱스를 동기화 위원회 듀티에 매핑

pub sync_duties: SyncDutiesMap,

/// 로컬에서 관리되는 검증자들의 정규 목록 제공

pub validator_store: Arc<S>,

/// 알려지지 않은 검증자 공개키를 다음 폴링 슬롯에 매핑
```

```
pub unknown_validator_next_poll_slots: RwLock<HashMap<PublicKeyBytes, Slot>>,
/// 현재 슬롯 추적
pub slot_clock: T,
/// 원격 비콘 노드에 대한 HTTP 접근 제공
pub beacon_nodes: Arc<BeaconNodeFallback<T>>,
/// 태스크 실행을 위한 런타임
pub executor: TaskExecutor,
/// 현재 체인 스펙
pub spec: Arc<ChainSpec>,
/// 대규모 검증자 수에 대한 메트릭 허용 여부
pub enable_high_validator_count_metrics: bool,
/// 분산 모드에서 실행 중인지 여부
pub distributed: bool,
/// 증명 비활성화 여부
pub disable_attesting: bool,
}
```

주요 타입 정의

```
/// 가독성을 위한 의존 루트 타입
type DependentRoot = Hash256;

/// 검증자 공개키 → 에포크별 듀티 매핑
```

```rust
type AttesterMap = HashMap<PublicKeyBytes, HashMap<Epoch, (DependentRoot, DutyAn
```

```rust
/// 에포크 → 제안자 데이터 매핑
type ProposerMap = HashMap<Epoch, (DependentRoot, Vec<ProposerData>)>;
```

DutyAndProof 구조체

```rust
#[derive(Clone)]
pub struct DutyAndProof {
pub duty: AttesterData,
/// 검증자가 집계자인 경우에만 Some으로 설정
pub selection_proof: Option<SelectionProof>,
/// 이 듀티에 대해 구독을 보낼 슬롯들 추적
pub subscription_slots: Arc<SubscriptionSlots>,
}
```

서비스 초기화 및 시작

DutiesServiceBuilder 패턴

```rust
// validator_client/src/lib.rs에서의 초기화
let duties_service = Arc::new(
DutiesServiceBuilder::new()
```

```
.slot_clock(slot_clock.clone())
.beacon_nodes(beacon_nodes.clone())
.validator_store(validator_store.clone())
.spec(context.eth2_config.spec.clone())
.executor(context.executor.clone())
.enable_high_validator_count_metrics(config.enable_high_validator_count_metrics)
.distributed(config.distributed)
.disable_attesting(config.disable_attesting)
.build()?,
);
```

업데이트 서비스 시작

`start_update_service` 함수는 여러 개의 백그라운드 태스크를 생성하여 듀티 관리를 수행합니다:

```
pub fn start_update_service<S: ValidatorStore + 'static, T: SlotClock + 'stati
core_duties_service: Arc<DutiesService<S, T>>,
mut block_service_tx: Sender<BlockServiceNotification>,
) {
// 1. 검증자 인덱스 업데이트 태스크
let duties_service = core_duties_service.clone();
core_duties_service.executor.spawn(
async move {
loop {
```

```
poll_validator_indices(&duties_service).await;

// 다음 슬롯까지 대기

if  let Some(duration) = duties_service.slot_clock.duration_to_next_slot() {

sleep(duration).await;

} else {

sleep(duties_service.slot_clock.slot_duration()).await;

}

}

},

"duties_service_indices",

);



// 2. 블록 제안 듀티 추적 태스크

let  duties_service = core_duties_service.clone();

core_duties_service.executor.spawn(

async  move {

loop {

if  let Some(duration) = duties_service.slot_clock.duration_to_next_slot() {

sleep(duration).await;

} else {

sleep(duties_service.slot_clock.slot_duration()).await;

continue;

}
```

```
if  let Err(e) = poll_beacon_proposers(&duties_service, &mut  block_service_tx).
error!(error = ?e, "Failed to poll beacon proposers")
}
}
},
"duties_service_proposers",
);

// 3. 증명 듀티 추적 태스크 (증명이 비활성화되지 않은 경우)
if !core_duties_service.disable_attesting {
let  duties_service = core_duties_service.clone();
core_duties_service.executor.spawn(
async  move {
loop {
if  let Some(duration) = duties_service.slot_clock.duration_to_next_slot() {
sleep(duration).await;
} else {
sleep(duties_service.slot_clock.slot_duration()).await;
continue;
}

if  let Err(e) = poll_beacon_attesters(&duties_service).await {
```

```
error!(error = ?e, "Failed to poll beacon attesters");
}
}
},
"duties_service_attesters",
);

// 4. 동기화 위원회 듀티 추적 태스크
let duties_service = core_duties_service.clone();
core_duties_service.executor.spawn(
async move {
loop {
if let Err(e) = poll_sync_committee_duties(&duties_service).await {
error!(error = ?e, "Failed to poll sync committee duties");
}

if let Some(duration) = duties_service.slot_clock.duration_to_next_slot() {
sleep(duration).await;
} else {
sleep(duties_service.slot_clock.slot_duration()).await;
continue;
}
}
```

```
},
"duties_service_sync_committee",
);
}
}
```

poll_beacon_attesters() 함수 상세 분석

함수 시그니처 및 목적

```
async fn poll_beacon_attesters<S: ValidatorStore + 'static, T: SlotClock + 'st
duties_service: &Arc<DutiesService<S, T>>,
) -> Result<(), Error<S::Error>>
```

이 함수는 비콘 노드에서 증명 듀티를 조회하고 다음 작업들을 순서대로 수행합니다:

1. 현재 에포크 듀티 폴링 및 로컬 `duties_service.attesters` 맵 업데이트
2. 다음 에포크에 대해서도 동일한 작업 수행
3. 비콘 노드에 증명 서브넷 구독 푸시
4. `duties_service.attesters` 에서 오래된 항목들 정리

핵심 구현 로직

```
async fn poll_beacon_attesters<S: ValidatorStore + 'static, T: SlotClock + 'st
duties_service: &Arc<DutiesService<S, T>>,
```

```
) -> Result<(), Error<S::Error>> {

// 현재 슬롯과 에포크 계산

let current_slot = duties_service

.slot_clock

.now()

.ok_or(Error::UnableToReadSlotClock)?;

let current_epoch = current_slot.epoch(S::E::slots_per_epoch());

let next_epoch = current_epoch + 1;



// 도플갱어 보호 중인 검증자들도 포함하여 모든 공개키 수집

// 도플갱어 검증자들의 듀티도 알아야 서브넷에 구독하고

// 다른 실행 인스턴스에 대한 정보를 얻을 수 있음

let local_pubkeys: HashSet<_> = duties_service

.validator_store

.voting_pubkeys(DoppelgangerStatus::ignored);



// 공개키에서 검증자 인덱스로 변환

let local_indices = {

let mut local_indices = Vec::with_capacity(local_pubkeys.len());

for &pubkey in &local_pubkeys {

if let Some(validator_index) = duties_service.validator_store.validator_index(&

local_indices.push(validator_index)

}
```

```
}
local_indices
};

// 현재 에포크 듀티 다운로드 및 업데이트
if  let Err(e) = poll_beacon_attesters_for_epoch(
duties_service,
current_epoch,
&local_indices,
&local_pubkeys,
).await {
error!(
%current_epoch,
request_epoch = %current_epoch,
err = ?e,
"Failed to download attester duties"
)
}

// 다음 에포크 듀티 다운로드 및 업데이트
if  let Err(e) = poll_beacon_attesters_for_epoch(
duties_service,
next_epoch,
```

```
&local_indices,
&local_pubkeys
).await {
error!(
%current_epoch,
request_epoch = %next_epoch,
err = ?e,
"Failed to download attester duties"
)
}

// 서브넷 구독 처리
// ... (구독 로직)

// 오래된 듀티 정리
// ... (정리 로직)

Ok(())
}
```

poll_beacon_attesters_for_epoch 함수

특정 에포크에 대한 듀티를 다운로드하고 저장하는 핵심 함수입니다:

```
async fn poll_beacon_attesters_for_epoch<S: ValidatorStore + 'static, T: SlotC
duties_service: &Arc<DutiesService<S, T>>,
epoch: Epoch,
local_indices: &[u64],
local_pubkeys: &HashSet<PublicKeyBytes>,
) -> Result<(), Error<S::Error>> {
// 검증자가 없으면 비콘 노드를 귀찮게 할 필요 없음
if local_indices.is_empty() {
debug!(%epoch, "No validators, not downloading duties");
return Ok(());
}


// 초기화되지 않은 검증자들에 대한 듀티 요청
// 없다면 INITIAL_DUTIES_QUERY_SIZE만큼의 검증자에 대해 요청
let uninitialized_validators =
get_uninitialized_validators(duties_service, &epoch, local_pubkeys);
let initial_indices_to_request = if !uninitialized_validators.is_empty() {
uninitialized_validators.as_slice()
} else {
&local_indices[0..min(INITIAL_DUTIES_QUERY_SIZE, local_indices.len())]
};
```

```
// 초기 요청 수행
let response = post_validator_duties_attester(
duties_service,
epoch,
initial_indices_to_request
).await?;
let dependent_root = response.dependent_root;


// 충돌하는 (epoch, dependent_root) 값이나 누락된 듀티를 가진 검증자들 찾기
let validators_to_update: Vec<_> = {
let attesters = duties_service.attesters.read();
local_pubkeys
.iter()
.filter(|pubkey| {
attesters.get(pubkey).is_none_or(|duties| {
duties
.get(&epoch)
.is_none_or(|(prior, _)| *prior != dependent_root)
})
})
.collect::<Vec<_>>()
};
```

```
if validators_to_update.is_empty() {
// 업데이트가 필요한 검증자가 없음
return Ok(());
}

// 추가 요청이 필요한 인덱스들에 대해 요청 수행
// ... (추가 요청 로직)

Ok(())
}
```

poll_beacon_proposers() 함수 상세 분석

함수 시그니처 및 목적

```
async fn poll_beacon_proposers<S: ValidatorStore, T: SlotClock + 'static>(
duties_service: &DutiesService<S, T>,
block_service_tx: &mut Sender<BlockServiceNotification>,
) -> Result<(), Error<S::Error>>
```

이 함수는 블록 제안 듀티를 관리하며 다음 작업들을 수행합니다:

1. 캐시된 제안자 정보를 사용하여 `BlockService` 에 초기 알림 전송
2. 비콘 노드에서 최신 제안자 듀티 다운로드

3. 새로운 제안자가 발견되면 추가 알림 전송

핵심 구현 로직

```
async fn poll_beacon_proposers<S: ValidatorStore, T: SlotClock + 'static>(
duties_service: &DutiesService<S, T>,
block_service_tx: &mut Sender<BlockServiceNotification>,
) -> Result<(), Error<S::Error>> {
let current_slot = duties_service
.slot_clock
.now()
.ok_or(Error::UnableToReadSlotClock)?;
let current_epoch = current_slot.epoch(S::E::slots_per_epoch());

// 캐시에 있는 제안자들에 대해 블록 제안 서비스에 알림
let initial_block_proposers = duties_service.block_proposers(current_slot);
notify_block_production_service::<S>(
current_slot,
&initial_block_proposers,
block_service_tx,
duties_service.validator_store.as_ref(),
).await;

// 도플갱어 보호 중인 검증자들도 포함하여 모든 공개키 수집
```

```
let local_pubkeys: HashSet<_> = duties_service
.validator_store
.voting_pubkeys(DoppelgangerStatus::ignored);

// 검증자가 있는 경우에만 듀티 다운로드 및 추가 블록 생성 이벤트 푸시
if !local_pubkeys.is_empty() {
let download_result = duties_service
.beacon_nodes
.first_success(|beacon_node| async move {
beacon_node
.get_validator_duties_proposer(current_epoch)
.await
})
.await;

match download_result {
Ok(response) => {
let dependent_root = response.dependent_root;

// 로컬 검증자들과 관련된 듀티만 필터링
let relevant_duties = response
.data
.into_iter()
```

```
.filter(|proposer_duty| local_pubkeys.contains(&proposer_duty.pubkey))
.collect::<Vec<_>>();


debug!(
%dependent_root,
num_relevant_duties = relevant_duties.len(),
"Downloaded proposer duties"
);


// 듀티 저장 및 재구성 감지
if  let Some((prior_dependent_root, _)) = duties_service
.proposers
.write()
.insert(current_epoch, (dependent_root, relevant_duties))
{
if  dependent_root != prior_dependent_root {
warn!(
%prior_dependent_root,
%dependent_root,
msg = "this may happen from time to time",
"Proposer duties re-org"
)
}
```

```
}
}
Err(e) => error!(err = %e, "Failed to download proposer duties"),
}

// 비콘 노드 업데이트 후 이 슬롯의 블록 제안자들을 다시 계산
let  additional_block_producers = duties_service
.block_proposers(current_slot)
.difference(&initial_block_proposers)
.copied()
.collect::<HashSet<PublicKeyBytes>>();

// 이 슬롯에 대한 새로운 제안자가 있으면 알림 전송
if !additional_block_producers.is_empty() {
notify_block_production_service::<S>(
current_slot,
&additional_block_producers,
block_service_tx,
duties_service.validator_store.as_ref(),
).await;
}
}
```

```
Ok(())
}
```

주요 메서드 분석

검증자 정보 조회 메서드들

```
impl<S: ValidatorStore, T: SlotClock + 'static> DutiesService<S, T> {
/// 듀티 서비스에 알려진 총 검증자 수 반환
pub fn total_validator_count(&self) -> usize {
self.validator_store.num_voting_validators()
}

/// 주어진 에포크에서 제안해야 하는 검증자 수 반환
pub fn proposer_count(&self, epoch: Epoch) -> usize {
// 도플갱어 보호 측면에서 안전한 것으로 간주되는 검증자들만 수집
let signing_pubkeys: HashSet<_> = self
.validator_store
.voting_pubkeys(DoppelgangerStatus::only_safe);

self.proposers
.read()
.get(&epoch)
```

```
.map_or(0, |(_, proposers)| {
proposers
.iter()
.filter(|proposer_data| signing_pubkeys.contains(&proposer_data.pubkey))
.count()
})
}

/// 주어진 에포크에서 증명해야 하는 검증자 수 반환
pub fn attester_count(&self, epoch: Epoch) -> usize {
let signing_pubkeys: HashSet<_> = self
.validator_store
.voting_pubkeys(DoppelgangerStatus::only_safe);
self.attesters
.read()
.iter()
.filter_map(|(_, map)| map.get(&epoch))
.map(|(_, duty_and_proof)| duty_and_proof)
.filter(|duty_and_proof| signing_pubkeys.contains(&duty_and_proof.duty.pubkey))
.count()
}

/// 주어진 슬롯에서 제안하도록 할당된 검증자들의 공개키 반환
```

```
pub fn block_proposers(&self, slot: Slot) -> HashSet<PublicKeyBytes> {
let epoch = slot.epoch(S::E::slots_per_epoch());
let signing_pubkeys: HashSet<_> = self
.validator_store
.voting_pubkeys(DoppelgangerStatus::only_safe);

self.proposers
.read()
.get(&epoch)
.map(|(_, proposers)| {
proposers
.iter()
.filter(|proposer_data| {
proposer_data.slot == slot
&& signing_pubkeys.contains(&proposer_data.pubkey)
})
.map(|proposer_data| proposer_data.pubkey)
.collect()
})
.unwrap_or_default()
}

/// 주어진 슬롯에 대한 모든 증명 듀티 반환
```

```
pub fn attesters(&self, slot: Slot) -> Vec<DutyAndProof> {
let epoch = slot.epoch(S::E::slots_per_epoch());
let signing_pubkeys: HashSet<_> = self
.validator_store
.voting_pubkeys(DoppelgangerStatus::only_safe);


self.attesters
.read()
.iter()
.filter_map(|(_, map)| map.get(&epoch))
.map(|(_, duty_and_proof)| duty_and_proof)
.filter(|duty_and_proof| {
duty_and_proof.duty.slot == slot
&& signing_pubkeys.contains(&duty_and_proof.duty.pubkey)
})
.cloned()
.collect()
}
}
```

서브넷 구독 관리

SubscriptionSlots 구조체

```rust
pub struct SubscriptionSlots {
/// 슬롯 내림차순으로 정렬된 (slot, already_sent) 쌍들
slots: Vec<(Slot, AtomicBool)>,
/// 듀티 자체의 슬롯
duty_slot: Slot,
}

impl SubscriptionSlots {
/// 새로운 SubscriptionSlots 생성
fn new(duty_slot: Slot, current_slot: Slot) -> Arc<Self> {
let slots = ATTESTATION_SUBSCRIPTION_OFFSETS
.into_iter()
.filter_map(|offset| duty_slot.safe_sub(offset).ok())
// 아직 발생하지 않은 예약된 슬롯만 유지하여 만료된 구독 전송 방지
.filter(|scheduled_slot| *scheduled_slot > current_slot)
.map(|scheduled_slot| (scheduled_slot, AtomicBool::new(false)))
.collect();
Arc::new(Self { slots, duty_slot })
}

/// 주어진 슬롯에서 구독을 보내야 하는지 확인
fn should_send_subscription_at(&self, slot: Slot) -> bool {
```

```
slot + MIN_ATTESTATION_SUBSCRIPTION_LOOKAHEAD <= self.duty_slot

&& self

.slots

.iter()

.rev()

.any(|(scheduled_slot, already_sent)| {

slot >= *scheduled_slot && !already_sent.load(Ordering::Relaxed)

})

}



/// 주어진 슬롯에서의 성공적인 구독을 기록

fn  record_successful_subscription_at(&self, slot: Slot) {

for (scheduled_slot, already_sent) in  self.slots.iter().rev() {

if  slot >= *scheduled_slot {

already_sent.store(true, Ordering::Relaxed);

} else {

break;

}

}

}

}
```

에러 처리 및 복구 메커니즘

에러 타입 정의

```
#[derive(Debug)]
pub enum Error<T> {
UnableToReadSlotClock,
FailedToDownloadAttesters(String),
FailedToProduceSelectionProof(ValidatorStoreError<T>),
InvalidModulo(ArithError),
Arith(ArithError),
SyncDutiesNotFound(u64),
}
```

복구 전략

1. **슬롯 클록 읽기 실패**: 한 슬롯 동안 대기 후 재시도
2. **네트워크 요청 실패**: 에러 로깅 후 다음 슬롯에서 재시도
3. **듀티 다운로드 실패**: 캐시된 값 사용하여 계속 진행
4. **도플갱어 보호**: 안전하지 않은 검증자들은 듀티 수행에서 제외

성능 최적화

메트릭 및 모니터링

```
// 검증자별 메트릭 수집 여부 결정
pub fn per_validator_metrics(&self) -> bool {
```

```
self.enable_high_validator_count_metrics
|| self.total_validator_count() <= VALIDATOR_METRICS_MIN_COUNT
}
```

배치 처리 최적화

- **초기 쿼리 크기**: `INITIAL_DUTIES_QUERY_SIZE = 1` 로 설정하여 불필요한 데이터 전송 최소화
- **선택적 업데이트**: `dependent_root` 비교를 통해 변경된 듀티만 업데이트
- **구독 최적화**: 미리 계산된 오프셋을 사용하여 효율적인 서브넷 구독

다른 서비스와의 상호작용

BlockService와의 연동

```
// 블록 제안 알림 전송
notify_block_production_service::<S>(
current_slot,
&block_proposers,
block_service_tx,
duties_service.validator_store.as_ref(),
).await;
```

AttestationService와의 연동

`AttestationService` 는 `DutiesService` 를 의존성으로 받아 증명 듀티 정보를 조회합니다:

```
// validator_client/validator_services/src/attestation_service.rs

pub struct Inner<S, T> {

duties_service: Arc<DutiesService<S, T>>,

validator_store: Arc<S>,

slot_clock: T,

// ...

}
```

결론

`DutiesService` 는 Lighthouse 검증자 클라이언트의 핵심 구성 요소로서, 복잡한 듀티 관리 로직을 효율적으로 처리합니다. `poll_beacon_attesters()` 와 `poll_beacon_proposers()` 함수는 각각 증명 및 제안 듀티를 주기적으로 업데이트하며, 도플갱어 보호, 서브넷 구독 관리, 에러 복구 등의 고급 기능을 제공합니다.

이 서비스의 설계는 확장성, 안정성, 성능을 모두 고려하여 대규모 검증자 운영 환경에서도 효율적으로 동작할 수 있도록 구현되었습니다. 특히 비동기 처리, 배치 최적화, 그리고 다른 서비스들과의 느슨한 결합을 통해 전체 시스템의 견고성을 보장합니다.